

DigiBank Dynamic Security Analysis

Runtime security validation of the DigiBank API.

1. Why DAST is needed

Static analysis examines source code and configuration before execution. Dynamic analysis complements it by interacting with the application while it is running. This matters because a system can compile, pass unit tests, and still expose an API route without authentication, return excessive technical detail, accept invalid input, or handle sessions incorrectly.

DAST checks what an external client can actually observe, not only what the source code appears to intend.

The sequence is: establish runtime behavior, identify security weaknesses, apply targeted remediation, and replay the same scenarios to demonstrate the result.

2. Application under test

The external client does not call each business service directly. The API Gateway is the security boundary and the single entry point for the business API. Behind it, the customer, account, transaction, compliance, and notification services run as separate Compose services.

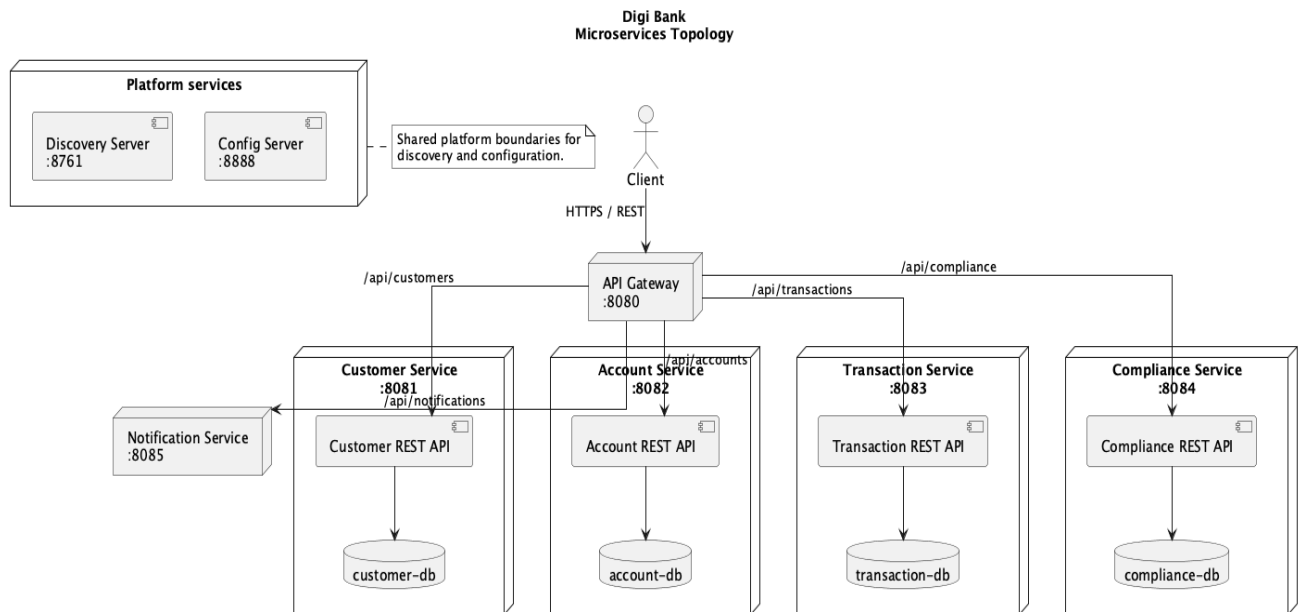


Figure 1. Runtime topology under test: gateway, business services, platform services, Compose host and persistence boundaries.

Surface	What is tested
Gateway	Authentication, authorization, headers, cache behavior, error responses and route protection.
Business APIs	Customer, account, transaction and compliance operations through gateway routing.
Runtime behavior	Malformed requests, invalid values, missing resources, session behavior and information disclosure.
Delivery path	Build, Compose startup, readiness, Newman replay and ZAP scanning.

3. Reproducible test environment

The environment is deliberately reproducible. Maven verifies the implementation, Docker Compose starts the complete topology, smoke checks verify the endpoints, and stack validation confirms that the expected containers are running before DAST begins.

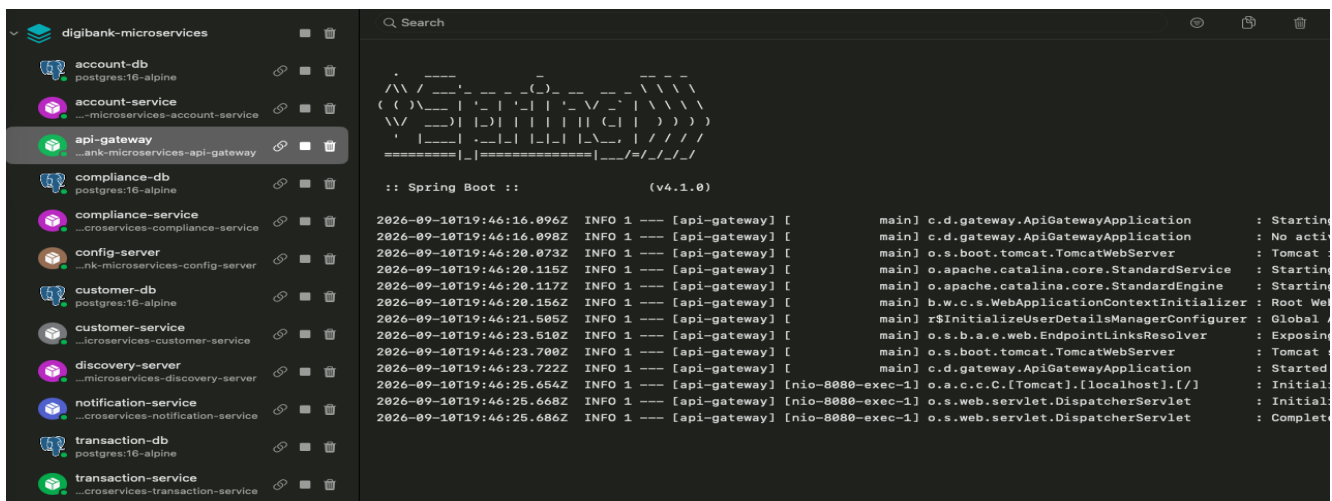
```
make clean
make build
make image-build
make up
make smoke
make validate-stack
make newman-scan
make zap-scan
```

3.1 Starting the application

The Compose startup creates the complete local runtime. The screenshot shows the command used to start the services and the resulting container state. This establishes the execution context before any security request is sent.

```
→ digibank-core git:(main) ✕ make up
/Applications/Xcode.app/Contents/Developer/usr/bin/make -C digibank-microservices up
docker compose -f docker-compose.yml up -d
[+] up 12/12
✓ Container digibank-microservices-config-server-1      Running      0.00s
✓ Container digibank-microservices-transaction-db-1    Healthy      1.25s
✓ Container digibank-microservices-compliance-db-1     Healthy      0.79s
✓ Container digibank-microservices-compliance-service-1 Running      0.00s
✓ Container digibank-microservices-account-db-1        Healthy      0.79s
✓ Container digibank-microservices-account-service-1   Running      0.00s
✓ Container digibank-microservices-notification-service-1 Running      0.00s
✓ Container digibank-microservices-transaction-service-1 Running      0.00s
✓ Container digibank-microservices-discovery-server-1  Running      0.00s
✓ Container digibank-microservices-customer-db-1       Healthy      0.79s
✓ Container digibank-microservices-customer-service-1  Running      0.00s
✓ Container digibank-microservices-api-gateway-1       Running      0.00s
```

The container view confirms that the gateway, business services, databases, discovery server and config server are present in the same runtime topology.



3.2 Smoke validation

Smoke validation checks the public health endpoint, gateway business routes, direct internal service endpoints, discovery registry and config server. It is not the security campaign; it proves that the application is alive and that later DAST failures can be interpreted as application behavior rather than startup failure.

```
→ digibank-core git:(main) ✗ make smoke
/Applications/Xcode.app/Contents/Developer/usr/bin/make -C digibank-microservices smoke
./scripts/smoke-test.sh
Checking gateway-health http://localhost:8080/actuator/health
Checking customer-api http://localhost:8080/api/customers
Checking account-api http://localhost:8080/api/accounts
Checking transaction-api http://localhost:8080/api/transactions
Checking compliance-api http://localhost:8080/api/compliance
Checking customer-service http://localhost:8081/api/customers
Checking account-service http://localhost:8082/api/accounts
Checking transaction-service http://localhost:8083/api/transactions
Checking compliance-service http://localhost:8084/api/compliance
Checking notification-service http://localhost:8085/actuator/health
Checking discovery-registry http://localhost:8761/registry
Checking config-server http://localhost:8888/config/customer-service
Microservice smoke checks passed.
```

The successful result is the gate before Newman and ZAP.

4. Baseline and remediation decision

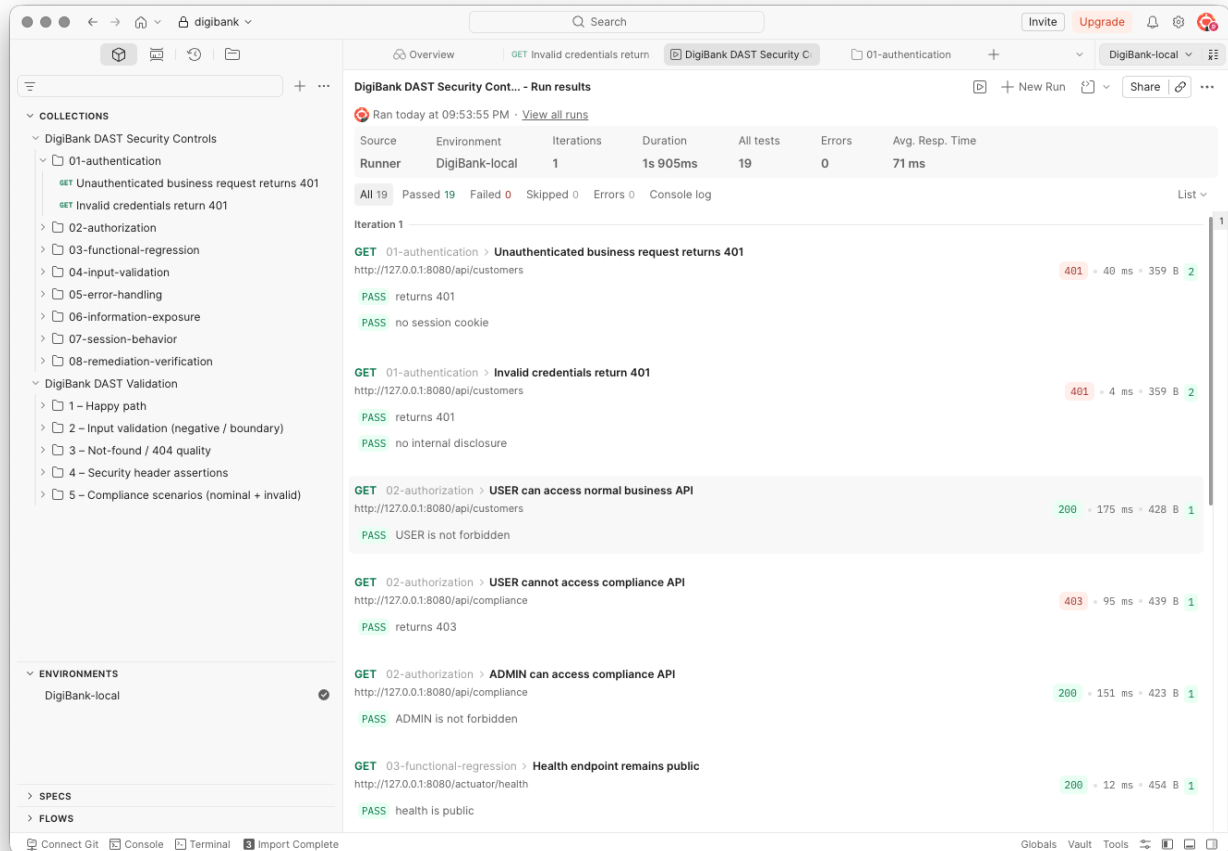
The baseline establishes the behavior before the gateway security boundary was introduced. The important observations were unauthenticated business routes, no gateway role distinction, incomplete response hardening, and no explicit cache policy for sensitive API responses.

The remediation uses stateless HTTP Basic authentication at the gateway because it is simple to reproduce in the workshop and preserves the existing service topology. It is not presented as the production identity architecture.

Request condition	Expected result after remediation
No credentials or invalid credentials	401 Unauthorized
Valid USER on normal banking API	Request is permitted
Valid USER on compliance API	403 Forbidden
Valid ADMIN on compliance API	Request is permitted
Authenticated request	No server session or session cookie

4.1 Authentication and authorization in Postman

The security collection makes the access rules visible through actual requests. The run shows unauthenticated requests returning 401, normal USER access returning 200, USER access to compliance returning 403, and ADMIN access to compliance returning 200.



The request results correspond directly to the gateway rules: authentication establishes identity, and authorization decides whether that identity may use the route.

4.2 Validation, errors and information exposure

The security collection also checks malformed JSON and missing resources. These cases return controlled 4xx responses without stack traces or internal implementation details. The same collection verifies the response headers on the health endpoint.

The screenshot displays the DigiBank DAST Security Controls interface. The left sidebar shows a tree view of collections and environments. The main panel shows the results of a test run titled 'DigiBank DAST Security Controls - Run results'. The test run was executed today at 09:53:55 PM. The summary table indicates that all 19 tests passed, with 0 errors and an average response time of 71 ms.

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	DigiBank-local	1	1s 905ms	19	0	71 ms

The test results are listed below the summary table. Each test entry includes the test name, the endpoint, the status, and the response details.

- GET /127.0.0.1:8080/api/customers**: **Invalid credentials return 401**. Status: **PASS**. Response: 401 - 92 ms - 447 B.
- POST /127.0.0.1:8080/api/customers**: **Malformed JSON is rejected without disclosure**. Status: **PASS**. Response: 400 - 92 ms - 447 B.
- GET /127.0.0.1:8080/api/customers/999999999**: **Missing resource uses controlled response**. Status: **PASS**. Response: 404 - 110 ms - 496 B.
- GET /127.0.0.1:8080/actuator/health**: **Health response has security headers**. Status: **PASS**. Response: 200 - 4 ms - 454 B.
- GET /127.0.0.1:8080/api/customers**: **Authenticated request is stateless**. Status: **PASS**. Response: 200 - 158 ms - 428 B.
- GET /127.0.0.1:8080/api/customers**: **Authentication remains enforced after remediation**. Status: **PASS**. Response: 401 - 4 ms - 359 B.

The run shows the validation and error-handling assertions passing, including the controlled 400 and 404 responses.

4.3 Headers and stateless behavior

The headers check verifies browser and response protections. The session check verifies that an authenticated request does not create a server session or return a session cookie.

The top screenshot displays the DigiBank DAST Security Controls interface for the test "Health response has security headers". The test is a GET request to `{{baseUrl}}/actuator/health`. The response is a 200 OK status with a body containing JSON data:

```
1 {
2   "groups": [
3     "liveness",
4     "readiness"
5   ],
6   "status": "UP"
7 }
```

The bottom screenshot displays the DigiBank DAST Security Controls interface for the test "Authenticated request is stateless". The test is a GET request to `{{baseUrl}}/api/customers`. The response is a 200 OK status with a body containing JSON data:

```
1 [
2   {
3     "id": 1,
4     "firstName": "DAST",
5     "lastName": "Customer",
6     "email": "dast.1789069699173@example.com"
7   },
8   {
9     "id": 2,
10    "firstName": "DAST",
11    "lastName": "Customer",
12    "email": "dast.1789073822431@example.com"
13  }
14 ]
```

These screenshots show the two related controls: hardened responses and stateless authentication.

5. Runtime checks and scan results

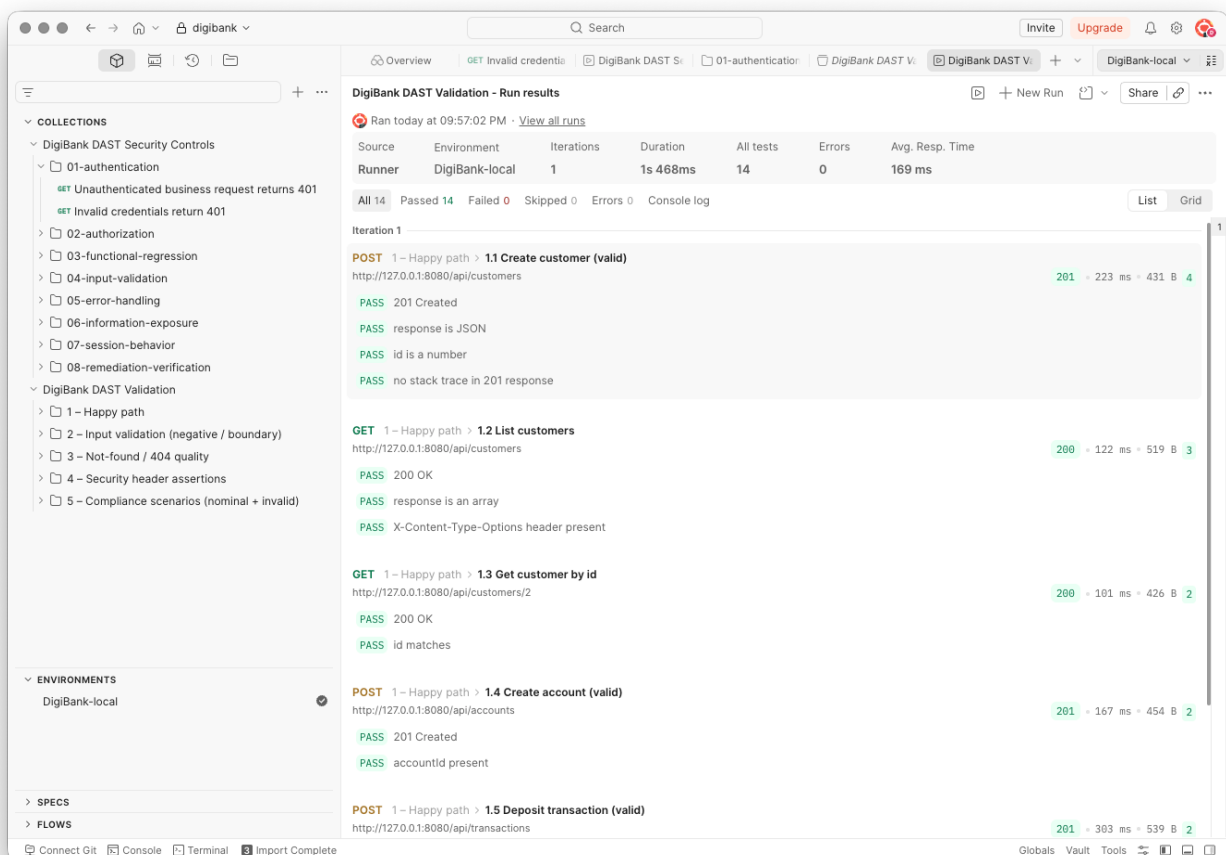
Newman expresses the expected security behavior as repeatable assertions. The security collection is divided into authentication, authorization, functional regression, input validation, error handling, information exposure, session behavior, and remediation verification.

Campaign	Result
Functional Newman collection	24 requests, 74 assertions, 0 failures.
Security Newman collection	12 requests, 19 assertions, 0 failures.
OWASP ZAP baseline	0 High, 1 Medium, 0 Low, 1 Informational.

The remaining Medium ZAP alert is the expected local warning that HTTP Basic credentials are sent over plain HTTP. It is acceptable only in this local educational environment and requires TLS in production. The Informational observation concerns public static-resource cacheability; authenticated API responses receive Cache-Control: no-store, private.

5.1 Functional regression in Postman

The functional collection checks the banking flows after the security changes: customer creation and retrieval, account creation, transactions and compliance operations. This confirms that the gateway protection did not break normal application behavior.



The collection run shows the banking requests and their passing assertions in the local environment.

5.2 Newman security replay

Newman repeats the same security collection without relying on the Postman interface. The summary is the reproducible command-line result used locally and in CI.

```
08-remediation-verification
Authentication remains enforced after remediation
GET http://127.0.0.1:8080/api/customers [401 Unauthorized, 359B, 6ms]
✓ authentication remains enforced
✓ no internal disclosure
```

	executed	failed
iterations	1	0
requests	12	0
test-scripts	12	0
prerequisite-scripts	0	0
assertions	19	0

```
total run duration: 1074ms
total data received: 1.24kB (approx)
average response time: 64ms [min: 4ms, max: 200ms, s.d.: 59ms]
[INFO] Functional report: /Users/adorsys123/dev/digibank-core/dast/reports/202609
5/functional-newman-report.json
```

The security replay completed 12 requests and 19 assertions with zero failures.

Directory listing for /

- [functional-newman-report.json](#)
- [security-newman-report.json](#)

The report directory contains separate functional and security JSON results.

5.3 OWASP ZAP scan

ZAP scans the externally visible gateway surface and complements the targeted Postman assertions. The scan checks the running application for observable response, header and authentication weaknesses that are not covered by a small set of targeted requests.

The scan completed with no High or Low alerts. The remaining findings are reviewed below and are interpreted against the local test environment rather than treated as an unexplained score.

5.4 ZAP alert summary

The ZAP summary reports zero High alerts, one Medium alert, zero Low alerts and one Informational alert. The remaining Medium alert is the local plain-HTTP warning associated with HTTP Basic authentication. It is recorded and explained in the report.

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	1
Low	0
Informational	1
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Info	Informational	http://127.0.0.1:8080	Percentage of responses with status code 2xx	50 %
Info	Informational	http://127.0.0.1:8080	Percentage of responses with status code 4xx	50 %
Info	Informational	http://127.0.0.1:8080	Percentage of endpoints with content type application/json	33 %
Info	Informational	http://127.0.0.1:8080	Percentage of endpoints with method GET	100 %
Info	Informational	http://127.0.0.1:8080	Count of total endpoints	3

6. CI validation

The CI workflow repeats the same validation chain after the application is built. Each step has a clear purpose: start the runtime, prove readiness, verify the exposed routes, replay the security assertions, and scan the gateway surface.

Pipeline step	Purpose
Build and tests	Compile the services and run the automated test suite.
Compose and readiness	Start the complete runtime and wait for the required endpoints.
Smoke validation	Confirm gateway, business APIs and platform services respond.
Newman	Run the versioned functional and security request assertions.
ZAP	Run an independent baseline scan against the running gateway.
Report upload	Keep Newman and ZAP results available when a check fails.

Newman and ZAP remain separate checks: Newman verifies expected security behavior, while ZAP performs broader automated observations of the exposed HTTP surface.

7. Deliverable mapping

Objective	Demonstrated by
A3.1	SAST/DAST explanation and runtime validation method.
A3.2	Versioned Postman functional and security collections.
A3.3	Assertions for status codes, headers, response bodies and error messages.
A3.4	Basic authentication, role checks and stateless-session checks.
A3.5	Baseline Newman behavior and ZAP observations.
A3.6	Gateway authentication, authorization, headers, cache and error hardening.
A3.7	Replayed Newman and ZAP validation after remediation.
A3.8	Technical report, reports, screenshots and CI artifacts.

The result is a documented chain from runtime observation, to risk interpretation, to remediation, to reproducible revalidation.